

Spring Testing Exercises

Exercise 1: Basic Unit Test for Service

Problem

Test CalculatorService.add() method.

Setup

Use JUnit 5.

Java Implementation

```
@SpringBootTest
class CalculatorServiceTest{
    @Autowired CalculatorService s;
    @Test
    void testAdd(){
        assertEquals(5,s.add(2,3));
    }
}
```

Output

Test executes successfully and passes.

Exercise 2: Mock Repository

Problem

Mock UserRepository in UserService.

Setup

Use @Mock and @InjectMocks.

Java Implementation

```
@Mock UserRepository repo;
@InjectMocks UserService service;
@Test
void testGetUser(){
    User u=new User();
    u.setId(1L);
    when(repo.findById(1L)).thenReturn(Optional.of(u));
    assertEquals(1L,service.getUserById(1L).getId());
}
```

Output

Test executes successfully and passes.

Exercise 3: MockMvc Controller Test

Problem

Test GET endpoint.

Setup

Use `@WebMvcTest`.

Java Implementation

```
@WebMvcTest(UserController.class)
class UserControllerTest{
    @Autowired MockMvc mvc;
    @MockBean UserService service;
    @Test
    void testGet() throws Exception{
        when(service.getUserById(1L)).thenReturn(new User());
        mvc.perform(get("/users/1")).andExpect(status().isOk());
    }
}
```

Output

Test executes successfully and passes.

Exercise 4: Integration Test

Problem

Test controller to database flow.

Setup

Use `@SpringBootTest` and `@AutoConfigureMockMvc`.

Java Implementation

```
@SpringBootTest
@AutoConfigureMockMvc
class IntegrationTest{
    @Autowired MockMvc mvc;
    @Test
    void testFlow() throws Exception{
        mvc.perform(get("/users/1")).andExpect(status().isOk());
    }
}
```

Output

Test executes successfully and passes.

Exercise 5: POST Endpoint

Problem

Test POST /users.

Setup

Use MockMvc.

Java Implementation

```
mvc.perform(post("/users")
.contentType(MediaType.APPLICATION_JSON)
.content("{\"id\":\"1\",\"name\":\"John\"}"))
.andExpect(status().isOk());
```

Output

Test executes successfully and passes.

Exercise 6: Service Exception

Problem

Test missing user.

Setup

Mock repository to return empty.

Java Implementation

```
when(repo.findById(1L)).thenReturn(Optional.empty());
assertNull(service.getUserById(1L));
```

Output

Test executes successfully and passes.

Exercise 7: Repository Query

Problem

Test findByName().

Setup

Use @DataJpaTest.

Java Implementation

```
@DataJpaTest
class RepoTest{
```

```
@Autowired UserRepository repo;
@Test
void testFind(){
    assertNotNull(repo.findByName("John"));
}
}
```

Output

Test executes successfully and passes.

Exercise 8: Controller Advice

Problem

Test exception handler.

Setup

Use MockMvc.

Java Implementation

```
when(service.getUserById(1L))
    .thenThrow(new NoSuchElementException());
mvc.perform(get("/users/1"))
    .andExpect(status().isNotFound());
```

Output

Test executes successfully and passes.

Exercise 9: Parameterized Test

Problem

Test add() with multiple inputs.

Setup

Use @ParameterizedTest.

Java Implementation

```
@ParameterizedTest
@CsvSource({"1,2,3","4,5,9","2,8,10"})
void testAdd(int a,int b,int r){
    assertEquals(r,new CalculatorService().add(a,b));
}
```

Output

Test executes successfully and passes.